

# Feature Implementation Requirements Report

**Prepared for:** LC Monitor Development Team

**Objective:** Implement 15-minute screenshot intervals and browser history tracking.

## 1. Architectural Context

---

LC Monitor is composed of three entirely separate parts that work together:

1. **The Web Dashboard** (Built with Lovable / React) - Displays the data.
2. **The Database & API** (Supabase) - Stores and routes the data.
3. **The Chrome Extension** (ID: knficjgnnobghcolkkhldjidamomnhec) - Collects the data.

Because modern web browsers physically block websites from accessing a user's screen or reading their browser history for security reasons, **all data collection relies entirely on the Chrome Extension.**

Therefore, modifying the web dashboard (Lovable code) alone is insufficient to build these features. We must modify the Extension.

## 2. Required Assets & Credentials

---

To move forward with implementing these two features, the following assets must be obtained from the original developer (Mantasha):

### 1. Chrome Extension Source Code (Critical)

**What it is:**

A folder containing files like manifest.json, background.js, and content.js.

**Why it is needed:**

We need to rewrite the extension's background timer to take screenshots exactly every 15 minutes, and add the chrome.history API to collect visited URLs.

### 2. Supabase Access (Required for History Feature)

**What it is:**

Dashboard access to the Supabase project aemljalkqrcucaxqnpv.

**Why it is needed:**

While screenshots already have a database table, "Browser History" is a brand new feature. We must create a new browser\_history table and set up Row Level Security (RLS) policies so the extension can safely save URLs to the database.

### 3. Chrome Web Store Access (Deployment)

#### What it is:

Access to the Google Developer account where the extension is published.

#### Why it is needed:

Once we modify the extension code, you must upload the new version to the Web Store so your employees receive the update automatically. (If employees installed it manually via developer mode, this is not required, but you will have to manually distribute the new folder to them).

## 3. Implementation Roadmap

Once the assets above are provided, here is exactly what we will execute across the three environments:

### Phase 1: Chrome Extension Updates

- **Screenshots:** Locate the current screenshot trigger logic in background.js and modify the chrome.alarms or setInterval logic to execute every 15 minutes.
- **History Tracking:**
  1. Add the "history" and "tabs" permissions to the manifest.json.
  2. Write an event listener that captures URLs when a user changes tabs or navigates to a new page.
  3. Write a batching function to send these URLs to the Supabase backend every few minutes to save battery and network bandwidth.

### Phase 2: Supabase Backend Updates

- **Migrations:** Run an SQL migration to create the browser\_history table (Fields: id, user\_id, url, title, duration\_seconds, timestamp).
- **Edge Functions:** Create a new Edge Function (or update admin/index.ts) that safely receives the history payload from the Chrome Extension and inserts it into the new table.

### Phase 3: Web Dashboard (Lovable) Updates

- **UI/UX:** Build a new "Browser History" view inside the Admin Dashboard.

- **API Integration:** Connect the React frontend to the new Supabase tables so managers can view a timeline of visited websites, filter by employee, and filter by date.